

SUBMITTED BY: HARSHPREET KAUR (harshpreet.kaur@kenexai.com)

DATE OF SUBMISSION: 5 JUNE, 2026

› Question 1: Income Tax Calculator (Basic Operations)

↳ 2 cells hidden

› Question 2: Student Final Grade Calculator (Global & Local Variables)

↳ 2 cells hidden

› Question 3: String Slicing Operations

↳ 2 cells hidden

› Question 4: Collection Types - Inventory Management System

↳ 2 cells hidden

› Question 5 Grade Classification System (Conditions)

↳ 2 cells hidden

› Question 6 Password Validation System (Iterations)

↳ 2 cells hidden

Question 7: Data Processing with Loops (ETL Simulation)

Description

Simulate an ETL (Extract, Transform, Load) process using loops to process transaction data.

Requirements

1. Create a list of transaction dictionaries with the following structure:

```
python { "transaction_id": 1, "product_id": 101, "amount": 200, "date": "2025-01-10" }
```

2. Use a for loop to iterate through transactions
3. Skip transactions with missing or invalid data (use continue)
4. Calculate total sales per product
5. Count valid and invalid transactions
6. Display summary report

Input/Output Example

```
=== ETL Data Processing ===
```

```
Processing transactions...
```

```
Skipping transaction 3 due to missing amount.
```

```
=== Summary Report ===
```

```
Total Transactions: 5
```

```
Valid Transactions: 4
```

```
Invalid Transactions: 1
```

```
Product Sales:
```

```
Product ID 101: Rs 450
```

```
Product ID 102: Rs 300
```

```
Product ID 103: Rs 150
```

Constraints

- Handle None values and missing keys
- Use continue for invalid transactions
- Use dictionaries to aggregate data

```
print("=== ETL Data Processing ===")  
print("Processing transactions...\n")
```

```

transactions = [
    {"transaction_id": 1, "product_id": 101, "amount": 200, "date": "2025-01-10"},
    {"transaction_id": 2, "product_id": 102, "amount": 300, "date": "2025-01-11"},
    {"transaction_id": 3, "product_id": 101, "amount": None, "date": "2025-01-12"},
    {"transaction_id": 4, "product_id": 103, "amount": 150, "date": "2025-01-13"},
    {"transaction_id": 5, "product_id": 101, "amount": 250, "date": "2025-01-14"}
]

total_transactions = len(transactions)
valid_transactions = 0
invalid_transactions = 0
product_sales = {}

for transaction in transactions:
    required_keys = ["transaction_id", "product_id", "amount", "date"]

    if not all(key in transaction for key in required_keys):
        print(f"Skipping transaction due to missing key: {transaction}")
        invalid_transactions += 1
        continue

    # Check invalid amount
    if transaction["amount"] is None or transaction["amount"] <= 0:
        print(
            f"Skipping transaction {transaction['transaction_id']} due to invalid amount"
        )
        invalid_transactions += 1
        continue

    # Extract
    product_id = transaction["product_id"]
    amount = transaction["amount"]

    # Transform + Load (Aggregation)
    if product_id in product_sales:
        product_sales[product_id] += amount
    else:
        product_sales[product_id] = amount

    valid_transactions += 1

print("\n=== Summary Report ===")
print(f"Total Transactions: {total_transactions}")
print(f"Valid Transactions: {valid_transactions}")
print(f"Invalid Transactions: {invalid_transactions}")

print("\nProduct Sales:")
for product_id, total_sales in product_sales.items():
    print(f"Product ID {product_id}: Rs {total_sales}")

```

```
=== ETL Data Processing ===  
Processing transactions...  
  
Skipping transaction 3 due to invalid amount.  
  
=== Summary Report ===  
Total Transactions: 5  
Valid Transactions: 4  
Invalid Transactions: 1  
  
Product Sales:  
Product ID 101: Rs 450  
Product ID 102: Rs 300  
Product ID 103: Rs 150
```

✓ Question 8: Pattern Matching - Data Format Processor

Description

Use structural pattern matching (match-case) to process different data formats.

Requirements

1. Create a function that processes different data types:
 - JSON format: {"type": "json", "content": "..."}
• CSV format: {"type": "csv", "content": "..."}
• Database format: {"type": "db", "content": {...}}
2. Use match-case to handle each format differently
3. Display appropriate processing message for each format
4. Handle unsupported formats

Input/Output Example

Processing: {'type': 'json', 'content': '{"name": "John"}'}

Output: Processing JSON data: {"name": "John"}

Processing: {'type': 'csv', 'content': 'name,age\nJohn,30'}

Output: Processing CSV data: name,age

John,30

Processing: {'type': 'db', 'content': {'name': 'John', 'age': 30}}

Output: Processing Database Record: {'name': 'John', 'age': 30}

Processing: {'type': 'xml', 'content': 'John'}

Output: Unsupported data format: xml

Constraints

- Use Python 3.10+ match-case syntax
- Handle all cases including default case
- Validate input structure

```
def process_data(data):
    if not isinstance(data, dict):
        print("Invalid input: Expected a dictionary.")
        return
    if "type" not in data or "content" not in data:
        print("Invalid input structure. Missing 'type' or 'content'.")
        return

    match data:

        case {"type": "json", "content": content}:
            print(f"Processing JSON data: {content}")

        case {"type": "csv", "content": content}:
            print(f"Processing CSV data: {content}")

        case {"type": "db", "content": content}:
            print(f"Processing Database Record: {content}")

        case {"type": format_type, "content": _}:
            print(f"Unsupported data format: {format_type}")

        case _:
            print("Invalid data structure")

# Test Data
data1 = {
    "type": "json",
    "content": '{"name": "John"}'
}

data2 = {
    "type": "csv",
    "content": "name,age\nJohn,30"
}

data3 = {
    "type": "db",
    "content": {
        "name": "John",
        "age": 30
    }
}
```

```
data4 = {  
    "type": "xml",  
    "content": "<name>John</name>"  
}  
  
# Processing  
for item in [data1, data2, data3, data4]:  
    print(f"\nProcessing: {item}")  
    process_data(item)
```

```
Processing: {'type': 'json', 'content': '{"name": "John"}'}
```

```
Processing JSON data: {"name": "John"}
```

```
Processing: {'type': 'csv', 'content': 'name,age\nJohn,30'}
```

```
Processing CSV data: name,age
```

```
John,30
```

```
Processing: {'type': 'db', 'content': {'name': 'John', 'age': 30}}
```

```
Processing Database Record: {'name': 'John', 'age': 30}
```

```
Processing: {'type': 'xml', 'content': '<name>John</name>'}
```

```
Unsupported data format: xml
```

> Question 9: String Manipulation - Text Analyzer

↳ 2 cells hidden

> Question 10: Comprehensive Calculator with All Concepts

↳ 2 cells hidden